

API Documentation
Algorithmic Trading System

Document Control

Document Title: API Documentation for Algorithmic Trading System

Version: 1.0

Date: April 24, 2025

Author: Senior Software Architect, Hedge Fund

Status: Draft

Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders

Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Software Architect	Initial draft of API Documentation for ATS

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Lead Developer	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Audience
 - 1.4. Assumptions and Constraints
 - 1.5. Definitions and Acronyms
- 2. [API Overview](#)
 - 2.1. API Purpose
 - 2.2. Base URL
 - 2.3. Authentication
 - 2.4. Rate Limiting
 - 2.5. Error Handling
 - 2.6. Response Format
- 3. [API Endpoints](#)
 - 3.1. Market Data Endpoints
 - 3.1.1. GET /market-data
 - 3.1.2. GET /market-data/historical
 - 3.1.3. WebSocket /ws/market-data
 - 3.2. Order Management Endpoints
 - 3.2.1. POST /orders
 - 3.2.2. GET /orders
 - 3.2.3. PATCH /orders/{order_id}
 - 3.2.4. DELETE /orders/{order_id}
 - 3.3. Portfolio Endpoints
 - 3.3.1. GET /portfolio
 - 3.3.2. GET /portfolio/performance
 - 3.4. Strategy Endpoints
 - 3.4.1. GET /strategies
 - 3.4.2. POST /strategies/signals
 - 3.5. Risk Management Endpoints
 - 3.5.1. GET /risk/parameters
 - 3.5.2. PATCH /risk/parameters
 - 3.6. Reporting Endpoints

- 3.6.1. GET /reports/trade-performance
- 3.6.2. GET /reports/portfolio-analytics
- 3.6.3. GET /reports/backtesting
- 3.6.4. GET /reports/compliance
- 3.7. Watchlist Endpoints
 - 3.7.1. GET /watchlist
 - 3.7.2. POST /watchlist
 - 3.7.3. DELETE /watchlist/{ticker}
- 3.8. Technical Analysis Endpoints
 - 3.8.1. GET /technical-analysis
- 3.9. News and Events Endpoints
 - 3.9.1. GET /news
- 4. [WebSocket API](#)
 - 4.1. Connection Details
 - 4.2. Message Formats
 - 4.3. Subscription Management
 - 4.4. Error Handling
- 5. [Security Considerations](#)
 - 5.1. Authentication and Authorization
 - 5.2. Data Encryption
 - 5.3. Rate Limiting
 - 5.4. Input Validation
 - 5.5. Audit Logging
- 6. [Error Codes and Messages](#)
 - 6.1. Standard HTTP Status Codes
 - 6.2. Custom Error Codes
- 7. [Best Practices for API Usage](#)
 - 7.1. Authentication Best Practices
 - 7.2. Efficient Data Retrieval
 - 7.3. Handling Rate Limits
 - 7.4. Error Handling Strategies
- 8. [Missing Information and Best Practices](#)
 - 8.1. Missing Information
 - 8.2. Best Practices Applied
- 9. [Implementation Notes](#)
 - 9.1. Development Environment
 - 9.2. Testing the API
 - 9.3. Deployment Considerations
- 10. [Glossary](#)
- 11. [Appendices](#)
 - 11.1. Sample API Requests
 - 11.2. OpenAPI Specification
 - 11.3. References

1. Introduction

1.1. Purpose

The API Documentation for the Algorithmic Trading System (ATS) provides a comprehensive guide to the Application Programming Interface (API) that supports the system's functionality. This document details the API endpoints, WebSocket connections, request/response formats, authentication mechanisms, error handling, and security considerations. It serves as a reference for developers integrating with the ATS, ensuring seamless interaction with the system's market data, order management, portfolio tracking, trading strategies, risk management, and reporting capabilities. The documentation aims to facilitate efficient development, testing, and maintenance of client applications, including the Market Watch Dashboard, while ensuring compliance with financial regulations.

1.2. Scope

The ATS API enables interaction with a high-performance platform for algorithmic paper trading across multiple asset classes, including stocks, indices, forex, and commodities, on major global exchanges (NYSE, NASDAQ, LSE, NSE). Integrated with the Interactive Brokers (IBKR) Paper Trading API, the ATS leverages Polars, DuckDB, and AI agents to support automated trading. The API scope includes:

- **Market Data:** Real-time and historical OHLCV data, technical indicators.
- **Order Management:** Placement, modification, cancellation, and status tracking of simulated trades.
- **Portfolio Management:** Tracking positions, performance, and profit/loss (P&L).
- **Trading Strategies:** Signal generation for Trend-Following and Support and Resistance strategies.
- **Risk Management:** Configuration and monitoring of risk parameters.
- **Reporting:** Generation of trade performance, portfolio analytics, backtesting, and compliance reports.
- **Watchlist:** Management of user-defined scrip lists.
- **Technical Analysis:** Market outlook and indicator data.
- **News and Events:** Placeholder for future integration of scrip-specific news.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Cryptocurrency or non-traditional asset support.
- Mobile app-specific APIs (dashboard is web-based).
- Advanced AI model training endpoints.
- Premium third-party data source integration.

1.3. Audience

This documentation is intended for:

- **Developers:** Building client applications, including the Market Watch Dashboard, or integrating ATS with other systems.
- **Traders:** Using API endpoints for custom trading tools or automation.
- **Compliance Officers:** Reviewing API logs and compliance reports.
- **IT Administrators:** Managing API deployment, security, and monitoring.
- **Product Owners:** Understanding API capabilities for feature planning.

1.4. Assumptions and Constraints

Assumptions:

- Users have access to valid OAuth2 credentials for authentication.
- The IBKR Paper Trading API provides reliable data and execution.
- Clients have basic knowledge of REST and WebSocket APIs.
- The development team is proficient in Python, FastAPI, and React.
- The Lenovo Legion Pro 5 supports API performance requirements.

Constraints:

- **Rate Limits:** 100 requests/minute for REST endpoints, 50 requests/second for IBKR data, 100 messages/second for orders.
- **Deployment:** Local on Lenovo Legion Pro 5, no cloud hosting.
- **Security:** Must use TLS 1.3, OAuth2, and JWT.
- **Compliance:** Adhere to SEC, GDPR, MiFID II, SEBI regulations.
- **Timeline:** API development within 3-6 months.

1.5. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **FastAPI:** Python framework for building APIs.
- **JWT:** JSON Web Token.
- **OAuth2:** Authentication framework.

- **REST:** Representational State Transfer.
 - **WebSocket:** Protocol for real-time communication.
-

2. API Overview

2.1. API Purpose

The ATS API provides programmatic access to the system's core functionalities, enabling developers to build client applications, automate trading workflows, and retrieve data for analysis. The API supports:

- Real-time and historical market data retrieval.
- Order placement and management in the IBKR paper trading environment.
- Portfolio tracking and performance analysis.
- Trading strategy signal generation and execution.
- Risk parameter configuration and monitoring.
- Report generation for performance, analytics, and compliance.
- Watchlist management and technical analysis.

2.2. Base URL

Base URL: `https://ats.local:8000`

- Deployed locally on Lenovo Legion Pro 5.
- Uses HTTPS with TLS 1.3 for secure communication.
- Port 8000 is configured for FastAPI.

2.3. Authentication

The API uses **OAuth2 with JWT** for authentication and authorization:

- **Token Endpoint:** POST `/auth/token`
- **Request:**
 - {
 - "username": "string",
 - "password": "string"
 - }
- **Response:**
 - {
 - "access_token": "string",
 - "token_type": "bearer",
 - "expires_in": 3600
 - }
- **Usage:** Include token in the Authorization header:
- Authorization: Bearer `<access_token>`
- **MFA:** Required for sensitive endpoints (e.g., order placement).
- **Token Expiry:** 1 hour, refreshable via `/auth/refresh`.
- **Implementation:** Keycloak for OAuth2, JWT with HS256 algorithm.

2.4. Rate Limiting

- **REST Endpoints:** 100 requests/minute per user.
- **WebSocket:** 10 concurrent connections per user.
- **IBKR Constraints:**
 - Market data: 50 requests/second.
 - Orders: 100 messages/second.
- **Enforcement:** Implemented via FastAPI middleware.
- **Response on Limit Exceed:**
 - {
 - "error": "rate_limit_exceeded",
 - "message": "Too many requests, please try again later.",
 - "retry_after": 60
 - }

2.5. Error Handling

- **Format:** All errors return JSON responses with:
- {

- "error": "string",
- "message": "string",
- "details": "object"
- }
- **HTTP Status Codes:** See Section 6.1.
- **Logging:** Errors are logged with timestamp, user ID, and request details.
- **Retry Logic:** Clients should retry on 429 (rate limit) or 503 (temporary failure) with exponential backoff.

2.6. Response Format

- **Success Responses:** JSON with data and metadata:
- {
- "data": "object | array",
- "metadata": {
- "timestamp": "datetime",
- "request_id": "string",
- "total_count": "integer"
- }
- }
- **Error Responses:** See Section 2.5.
- **Content-Type:** application/json.
- **Encoding:** UTF-8.

3. API Endpoints

3.1. Market Data Endpoints

3.1.1. GET /market-data

Description: Retrieves real-time OHLCV data for a specified ticker.

Parameters:

- **ticker** (query, string, required): Ticker symbol (e.g., "AAPL").
- **timeframe** (query, string, required): Timeframe (e.g., "1m", "5m", "1h", "1d").
- **exchange** (query, string, optional): Exchange (e.g., "NASDAQ"). Defaults to primary exchange.

Request:

GET /market-data?ticker=AAPL&timeframe=1m&exchange=NASDAQ

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "ticker": "AAPL",
      "exchange": "NASDAQ",
      "timestamp": "2025-04-24T10:00:00Z",
      "open": 150.25,
      "high": 151.10,
      "low": 149.80,
      "close": 150.90,
      "volume": 1000000
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12345",
    "total_count": 1
  }
}
```

Errors:

- **400 Bad Request:** Invalid ticker or timeframe.

- **401 Unauthorized:** Missing or invalid token.
- **429 Too Many Requests:** Rate limit exceeded.

Notes:

- Data is sourced from IBKR API via `ib_insync`.
- Maximum 100 records returned per request.
- Use WebSocket for continuous updates.

3.1.2. GET /market-data/historical

Description: Retrieves historical OHLCV data for a specified ticker.

Parameters:

- **ticker** (query, string, required): Ticker symbol.
- **timeframe** (query, string, required): Timeframe.
- **start_date** (query, string, required): Start date (ISO 8601, e.g., "2025-01-01").
- **end_date** (query, string, optional): End date. Defaults to current date.
- **exchange** (query, string, optional): Exchange.

Request:

GET /market-data/historical?ticker=AAPL&timeframe=1d&start_date=2025-01-01&end_date=2025-04-24

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "ticker": "AAPL",
      "exchange": "NASDAQ",
      "timestamp": "2025-01-01T00:00:00Z",
      "open": 145.00,
      "high": 147.50,
      "low": 144.20,
      "close": 146.80,
      "volume": 5000000
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12346",
    "total_count": 100
  }
}
```

Errors:

- **400 Bad Request:** Invalid date format or timeframe.
- **401 Unauthorized:** Invalid token.
- **429 Too Many Requests:** Rate limit exceeded.

Notes:

- Data stored in DuckDB, sourced from IBKR.
- Maximum 1,000 records per request.
- Use pagination for large datasets.

3.1.3. WebSocket /ws/market-data

Description: Streams real-time OHLCV data for subscribed tickers.

Connection:

wss://ats.local:8000/ws/market-data

Authorization: Bearer <access_token>

Subscription Message:

```
{
  "action": "subscribe",
  "tickers": ["AAPL", "MSFT"],
}
```

```
"timeframe": "1m"
}
```

Message Format:

```
{
  "ticker": "AAPL",
  "exchange": "NASDAQ",
  "timestamp": "2025-04-24T10:00:00Z",
  "open": 150.25,
  "high": 151.10,
  "low": 149.80,
  "close": 150.90,
  "volume": 1000000
}
```

Errors:

- **401 Unauthorized:** Invalid token.
- **400 Bad Request:** Invalid subscription message.

Notes:

- Updates every 5-10 seconds.
- Maximum 10 tickers per connection.
- Reconnect on disconnection with exponential backoff.

3.2. Order Management Endpoints

3.2.1. POST /orders

Description: Places a new order in the IBKR paper trading environment.

Body:

```
{
  "ticker": "string",
  "action": "string",
  "quantity": "integer",
  "price": "float",
  "type": "string",
  "account_id": "string"
}
```

- **ticker:** Ticker symbol (e.g., "AAPL").
- **action:** "BUY" or "SELL".
- **quantity:** Number of units.
- **price:** Target price for limit/stop orders.
- **type:** "MARKET", "LIMIT", "STOP", "BRACKET".
- **account_id:** IBKR account ID (e.g., "DUK221396").

Request:

POST /orders

Authorization: Bearer <access_token>

Content-Type: application/json

```
{
  "ticker": "AAPL",
  "action": "BUY",
  "quantity": 100,
  "price": 150.90,
  "type": "LIMIT",
  "account_id": "DUK221396"
}
```

Response (201 Created):

```
{
  "data": {
    "order_id": "O12345",
    "ticker": "AAPL",
    "action": "BUY",

```

```

    "quantity": 100,
    "price": 150.90,
    "type": "LIMIT",
    "status": "PENDING",
    "timestamp": "2025-04-24T10:00:00Z"
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12347"
  }
}

```

Errors:

- **400 Bad Request:** Invalid ticker, quantity, or order type.
- **401 Unauthorized:** Invalid token or MFA required.
- **429 Too Many Requests:** Rate limit exceeded.
- **422 Unprocessable Entity:** Insufficient funds or risk violation.

Notes:

- Orders are validated by Risk Management Module.
- Bracket orders include stop-loss and take-profit.

3.2.2. GET /orders

Description: Retrieves a list of orders.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **status** (query, string, optional): Filter by status (e.g., "PENDING", "FILLED").
- **ticker** (query, string, optional): Filter by ticker.
- **limit** (query, integer, optional): Maximum records (default: 50).
- **offset** (query, integer, optional): Pagination offset (default: 0).

Request:

GET /orders?account_id=DUK221396&status=PENDING&limit=10

Authorization: Bearer <access_token>

Response (200 OK):

```

{
  "data": [
    {
      "order_id": "O12345",
      "ticker": "AAPL",
      "action": "BUY",
      "quantity": 100,
      "price": 150.90,
      "type": "LIMIT",
      "status": "PENDING",
      "timestamp": "2025-04-24T10:00:00Z"
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12348",
    "total_count": 10
  }
}

```

Errors:

- **400 Bad Request:** Invalid account_id or parameters.
- **401 Unauthorized:** Invalid token.

Notes:

- Supports pagination for large datasets.

- Status updates via WebSocket.

3.2.3. PATCH /orders/{order_id}

Description: Modifies an existing order.

Parameters:

- **order_id** (path, string, required): Order ID.

Body:

```
{
  "quantity": "integer",
  "price": "float"
}
```

Request:

PATCH /orders/O12345

Authorization: Bearer <access_token>

Content-Type: application/json

```
{
  "quantity": 50,
  "price": 151.00
}
```

Response (200 OK):

```
{
  "data": {
    "order_id": "O12345",
    "ticker": "AAPL",
    "action": "BUY",
    "quantity": 50,
    "price": 151.00,
    "type": "LIMIT",
    "status": "PENDING",
    "timestamp": "2025-04-24T10:01:00Z"
  },
  "metadata": {
    "timestamp": "2025-04-24T10:01:05Z",
    "request_id": "req_12349"
  }
}
```

Errors:

- **400 Bad Request:** Invalid order_id or parameters.
- **401 Unauthorized:** Invalid token or MFA required.
- **404 Not Found:** Order not found.

Notes:

- Only pending orders can be modified.
- Risk Management validates changes.

3.2.4. DELETE /orders/{order_id}

Description: Cancels an existing order.

Parameters:

- **order_id** (path, string, required): Order ID.

Request:

DELETE /orders/O12345

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "order_id": "O12345",
    "status": "CANCELED"
  },
  "metadata": {
```

```
"timestamp": "2025-04-24T10:01:05Z",
"request_id": "req_12350"
}
}
```

Errors:

- **400 Bad Request:** Invalid order_id.
- **401 Unauthorized:** Invalid token.
- **404 Not Found:** Order not found.

Notes:

- Only pending orders can be canceled.
- Cancellation logged for compliance.

3.3. Portfolio Endpoints

3.3.1. GET /portfolio

Description: Retrieves current portfolio positions.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **ticker** (query, string, optional): Filter by ticker.
- **limit** (query, integer, optional): Maximum records (default: 50).
- **offset** (query, integer, optional): Pagination offset.

Request:

GET /portfolio?account_id=DUK221396

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "account_id": "DUK221396",
      "ticker": "AAPL",
      "quantity": 100,
      "entry_price": 150.90,
      "current_price": 151.50,
      "pnl": 60.00,
      "timestamp": "2025-04-24T10:00:00Z"
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12351",
    "total_count": 1
  }
}
```

Errors:

- **400 Bad Request:** Invalid account_id.
- **401 Unauthorized:** Invalid token.

Notes:

- P&L calculated as (current_price - entry_price) × quantity.
- Real-time updates via WebSocket.

3.3.2. GET /portfolio/performance

Description: Retrieves portfolio performance metrics.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **start_date** (query, string, required): Start date (ISO 8601).
- **end_date** (query, string, optional): End date (default: current date).

Request:

GET /portfolio/performance?account_id=DUK221396&start_date=2025-01-01

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "account_id": "DUK221396",
    "total_pnl": 500.00,
    "sharpe_ratio": 1.5,
    "max_drawdown": 0.10,
    "win_rate": 0.65,
    "equity_curve": [
      {"date": "2025-01-01", "value": 10000.00},
      {"date": "2025-04-24", "value": 10500.00}
    ]
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12352"
  }
}
```

Errors:

- **400 Bad Request:** Invalid date format.
- **401 Unauthorized:** Invalid token.

Notes:

- Metrics calculated using pandas.
- Equity curve limited to 1,000 points.

3.4. Strategy Endpoints

3.4.1. GET /strategies

Description: Retrieves available trading strategies.

Request:

GET /strategies

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "strategy_id": "trend_following",
      "name": "Trend-Following",
      "description": "Captures long-term price trends using VW SMA and MFI."
    },
    {
      "strategy_id": "support_resistance",
      "name": "Support and Resistance",
      "description": "Trades breakouts/reversals using Pivot Points and Fibonacci."
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12353",
    "total_count": 2
  }
}
```

Errors:

- **401 Unauthorized:** Invalid token.

Notes:

- Static list of strategies.
- Extensible for future strategies.

3.4.2. POST /strategies/signals

Description: Generates trade signals for a specified strategy.

Body:

```
{
  "strategy_id": "string",
  "ticker": "string",
  "timeframe": "string"
}
```

Request:

POST /strategies/signals

Authorization: Bearer <access_token>

Content-Type: application/json

```
{
  "strategy_id": "trend_following",
  "ticker": "AAPL",
  "timeframe": "1d"
}
```

Response (200 OK):

```
{
  "data": {
    "signal_id": "S12345",
    "strategy_id": "trend_following",
    "ticker": "AAPL",
    "action": "BUY",
    "price": 150.90,
    "timestamp": "2025-04-24T10:00:00Z"
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12354"
  }
}
```

Errors:

- **400 Bad Request:** Invalid strategy_id or ticker.
- **401 Unauthorized:** Invalid token.

Notes:

- Signals generated by Strategy Engine.
- Published to RabbitMQ for execution.

3.5. Risk Management Endpoints

3.5.1. GET /risk/parameters

Description: Retrieves current risk parameters.

Parameters:

- **account_id** (query, string, required): IBKR account ID.

Request:

GET /risk/parameters?account_id=DUK221396

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "account_id": "DUK221396",
    "max_risk_per_trade": 0.01,
    "max_open_risk": 0.10,
    "max_drawdown": 0.20,
    "circuit_breaker_threshold": 0.05
  },
  "metadata": {

```

```
"timestamp": "2025-04-24T10:00:05Z",
"request_id": "req_12355"
}
}
```

Errors:

- **400 Bad Request:** Invalid account_id.
- **401 Unauthorized:** Invalid token.

Notes:

- Parameters stored in DuckDB.
- Used by Risk Management Module.

3.5.2. PATCH /risk/parameters

Description: Updates risk parameters.

Parameters:

- **account_id** (query, string, required): IBKR account ID.

Body:

```
{
  "max_risk_per_trade": "float",
  "max_open_risk": "float",
  "max_drawdown": "float",
  "circuit_breaker_threshold": "float"
}
```

Request:

PATCH /risk/parameters?account_id=DUK221396

Authorization: Bearer <access_token>

Content-Type: application/json

```
{
  "max_risk_per_trade": 0.015,
  "max_open_risk": 0.12
}
```

Response (200 OK):

```
{
  "data": {
    "account_id": "DUK221396",
    "max_risk_per_trade": 0.015,
    "max_open_risk": 0.12,
    "max_drawdown": 0.20,
    "circuit_breaker_threshold": 0.05
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12356"
  }
}
```

Errors:

- **400 Bad Request:** Invalid parameters.
- **401 Unauthorized:** Invalid token or MFA required.

Notes:

- Updates validated for compliance.
- Logged for audit trail.

3.6. Reporting Endpoints

3.6.1. GET /reports/trade-performance

Description: Generates a trade performance report.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **start_date** (query, string, required): Start date.
- **end_date** (query, string, optional): End date.

- **format** (query, string, optional): Output format ("csv", "pdf", "png"). Default: "json".

Request:

GET /reports/trade-performance?account_id=DUK221396&start_date=2025-01-01&format=csv

Authorization: Bearer <access_token>

Response (200 OK, JSON):

```
{
  "data": {
    "total_trades": 100,
    "win_rate": 0.65,
    "sharpe_ratio": 1.5,
    "max_drawdown": 0.10,
    "pnl": 500.00
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12357"
  }
}
```

Response (200 OK, CSV):

total_trades,win_rate,sharpe_ratio,max_drawdown,pnl
100,0.65,1.5,0.10,500.00

Errors:

- **400 Bad Request:** Invalid date or format.
- **401 Unauthorized:** Invalid token.

Notes:

- Uses pandas for aggregation, Matplotlib for PNG.
- CSV/PDF files served via temporary URLs.

3.6.2. GET /reports/portfolio-analytics

Description: Generates portfolio analytics report.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **start_date** (query, string, required): Start date.
- **end_date** (query, string, optional): End date.
- **format** (query, string, optional): "csv", "pdf", "png". Default: "json".

Request:

GET /reports/portfolio-analytics?account_id=DUK221396&start_date=2025-01-01

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "exposure": {
      "stocks": 0.60,
      "forex": 0.30
    },
    "var_95": 1000.00,
    "correlations": {
      "AAPL_MSFT": 0.75
    }
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12358"
  }
}
```

Errors:

- **400 Bad Request:** Invalid parameters.

- **401 Unauthorized:** Invalid token.

Notes:

- Calculates VaR using historical simulation.
- Correlations computed with pandas.

3.6.3. GET /reports/backtesting

Description: Generates backtesting report for a strategy.

Parameters:

- **strategy_id** (query, string, required): Strategy ID.
- **ticker** (query, string, required): Ticker symbol.
- **start_date** (query, string, required): Start date.
- **end_date** (query, string, optional): End date.
- **format** (query, string, optional): "csv", "pdf", "png".

Request:

GET /reports/backtesting?strategy_id=trend_following&ticker=AAPL&start_date=2025-01-01

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "strategy_id": "trend_following",
    "ticker": "AAPL",
    "total_return": 0.12,
    "sharpe_ratio": 1.8,
    "max_drawdown": 0.08,
    "trades": 50
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12359"
  }
}
```

Errors:

- **400 Bad Request:** Invalid strategy_id or ticker.
- **401 Unauthorized:** Invalid token.

Notes:

- Uses historical data from DuckDB.
- Limited to 5 years of data.

3.6.4. GET /reports/compliance

Description: Generates compliance report with transaction logs.

Parameters:

- **account_id** (query, string, required): IBKR account ID.
- **start_date** (query, string, required): Start date.
- **end_date** (query, string, optional): End date.
- **format** (query, string, optional): "csv", "pdf".

Request:

GET /reports/compliance?account_id=DUK221396&start_date=2025-01-01

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "trade_id": "T12345",
      "timestamp": "2025-01-01T10:00:00Z",
      "ticker": "AAPL",
      "action": "BUY",
      "quantity": 100,

```

```

    "price": 150.90,
    "user_id": "user123"
  }
],
"metadata": {
  "timestamp": "2025-04-24T10:00:05Z",
  "request_id": "req_12360",
  "total_count": 100
}
}

```

Errors:

- **400 Bad Request:** Invalid parameters.
- **401 Unauthorized:** Invalid token.

Notes:

- Immutable logs for regulatory audits.
- Retained for 7 years.

3.7. Watchlist Endpoints

3.7.1. GET /watchlist

Description: Retrieves the user's watchlist.

Parameters:

- **user_id** (query, string, required): User ID.

Request:

GET /watchlist?user_id=user123

Authorization: Bearer <access_token>

Response (200 OK):

```

{
  "data": [
    {
      "ticker": "AAPL",
      "exchange": "NASDAQ",
      "added_at": "2025-04-24T10:00:00Z"
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12361",
    "total_count": 1
  }
}

```

Errors:

- **400 Bad Request:** Invalid user_id.
- **401 Unauthorized:** Invalid token.

Notes:

- Stored in DuckDB per user.
- Maximum 50 tickers per watchlist.

3.7.2. POST /watchlist

Description: Adds a ticker to the watchlist.

Body:

```

{
  "user_id": "string",
  "ticker": "string",
  "exchange": "string"
}

```

Request:

POST /watchlist

Authorization: Bearer <access_token>

Content-Type: application/json

```
{
  "user_id": "user123",
  "ticker": "AAPL",
  "exchange": "NASDAQ"
}
```

Response (201 Created):

```
{
  "data": {
    "ticker": "AAPL",
    "exchange": "NASDAQ",
    "added_at": "2025-04-24T10:00:00Z"
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12362"
  }
}
```

Errors:

- **400 Bad Request:** Invalid ticker or exchange.
- **401 Unauthorized:** Invalid token.
- **409 Conflict:** Ticker already in watchlist.

Notes:

- Validates ticker via IBKR API.
- Logs addition for audit.

3.7.3. DELETE /watchlist/{ticker}

Description: Removes a ticker from the watchlist.

Parameters:

- **ticker** (path, string, required): Ticker symbol.
- **user_id** (query, string, required): User ID.

Request:

DELETE /watchlist/AAPL?user_id=user123

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "ticker": "AAPL",
    "status": "removed"
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12363"
  }
}
```

Errors:

- **400 Bad Request:** Invalid ticker or user_id.
- **401 Unauthorized:** Invalid token.
- **404 Not Found:** Ticker not in watchlist.

Notes:

- Logs removal for audit.
- Updates watchlist in real-time.

3.8. Technical Analysis Endpoints

3.8.1. GET /technical-analysis

Description: Retrieves technical analysis data for a ticker.

Parameters:

- **ticker** (query, string, required): Ticker symbol.
- **timeframe** (query, string, required): Timeframe.
- **indicators** (query, string, optional): Comma-separated indicators (e.g., "VW_SMA,MFI"). Default: all.

Request:

GET /technical-analysis?ticker=AAPL&timeframe=1d&indicators=VW_SMA,MFI

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": {
    "ticker": "AAPL",
    "timeframe": "1d",
    "outlook": "Uptrend",
    "indicators": {
      "VW_SMA_55": 150.00,
      "MFI_14": 65.00
    }
  },
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12364"
  }
}
```

Errors:

- **400 Bad Request:** Invalid ticker or indicators.
- **401 Unauthorized:** Invalid token.

Notes:

- Outlook: "Strong Uptrend", "Uptrend", "Sideways", "Downtrend", "Strong Downtrend".
- Indicators calculated by Polars.

3.9. News and Events Endpoints

3.9.1. GET /news

Description: Placeholder for retrieving scrip-specific news and events.

Parameters:

- **ticker** (query, string, required): Ticker symbol.
- **start_date** (query, string, optional): Start date.
- **end_date** (query, string, optional): End date.

Request:

GET /news?ticker=AAPL&start_date=2025-04-01

Authorization: Bearer <access_token>

Response (200 OK):

```
{
  "data": [
    {
      "ticker": "AAPL",
      "title": "Apple AGM Scheduled",
      "date": "2025-04-25",
      "source": "Placeholder"
    }
  ],
  "metadata": {
    "timestamp": "2025-04-24T10:00:05Z",
    "request_id": "req_12365",
    "total_count": 1
  }
}
```

Errors:

- **400 Bad Request:** Invalid ticker or dates.
- **401 Unauthorized:** Invalid token.

Notes:

- Placeholder for future Reuters/Bloomberg integration.
- Limited to events within 10 days.

4. WebSocket API

4.1. Connection Details

- **Endpoint:** wss://ats.local:8000/ws/market-data
- **Protocol:** WebSocket over TLS 1.3.
- **Authentication:** Include JWT in header:
- Authorization: Bearer <access_token>
- **Connection Limit:** 10 concurrent connections per user.
- **Heartbeat:** Send ping every 30 seconds to maintain connection.

4.2. Message Formats

Subscription:

```
{
  "action": "subscribe",
  "tickers": ["string"],
  "timeframe": "string"
}
```

Unsubscription:

```
{
  "action": "unsubscribe",
  "tickers": ["string"]
}
```

Data Message:

```
{
  "ticker": "string",
  "exchange": "string",
  "timestamp": "datetime",
  "open": "float",
  "high": "float",
  "low": "float",
  "close": "float",
  "volume": "integer"
}
```

Error Message:

```
{
  "error": "string",
  "message": "string"
}
```

4.3. Subscription Management

- **Subscribe:** Add tickers to receive updates.
- **Unsubscribe:** Remove tickers from updates.
- **Maximum Tickers:** 10 per connection.
- **Validation:** Tickers validated via IBKR API.
- **Updates:** Sent every 5-10 seconds.

4.4. Error Handling

- **Invalid Token:** Connection closed with error message.
- **Invalid Tickers:** Error message sent, subscription ignored.
- **Rate Limit:** Connection throttled, error message sent.
- **Reconnection:** Clients should reconnect with exponential backoff (1s, 2s, 4s).

5. Security Considerations

5.1. Authentication and Authorization

- **OAuth2 with JWT:** Required for all endpoints.
- **MFA:** Mandatory for order management and risk parameter updates.
- **Role-Based Access:**
 - Traders: Access to market data, orders, portfolio.
 - Admins: Access to risk parameters, compliance reports.
- **Token Expiry:** 1 hour, refreshable.
- **Implementation:** Keycloak with HS256 JWT.

5.2. Data Encryption

- **In Transit:** TLS 1.3 for all API and WebSocket traffic.
- **At Rest:** AES-256 for sensitive data in DuckDB.
- **Credentials:** Stored in encrypted vault (HashiCorp Vault).

5.3. Rate Limiting

- **REST:** 100 requests/minute per user.
- **WebSocket:** 10 connections, 10 tickers per connection.
- **Implementation:** FastAPI middleware with Redis.

5.4. Input Validation

- **Schema Validation:** Pydantic models for request bodies.
- **Sanitization:** Remove invalid characters from inputs.
- **Constraints:** Enforce valid ranges (e.g., quantity > 0).

5.5. Audit Logging

- **Logged Actions:** All API calls, order placements, cancellations.
- **Fields:** Timestamp, user_id, endpoint, request_id, status.
- **Storage:** DuckDB, retained for 7 years.
- **Access:** Restricted to compliance officers.

6. Error Codes and Messages

6.1. Standard HTTP Status Codes

Code	Description	Example Message
200	OK	Request successful
201	Created	Resource created
400	Bad Request	Invalid request parameters
401	Unauthorized	Invalid or missing authentication token
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
422	Unprocessable Entity	Validation error
429	Too Many Requests	Rate limit exceeded, retry after 60s
500	Internal Server Error	Unexpected server error
503	Service Unavailable	Temporary service outage, retry later

6.2. Custom Error Codes

Code	Description	Example Message
invalid_ticker	Invalid ticker symbol	Ticker not recognized by IBKR
invalid_timeframe	Invalid timeframe	Timeframe must be 1m, 5m, 1h, etc.
insufficient_funds	Insufficient account balance	Not enough funds for order
risk_violation	Risk parameter violation	Order exceeds max risk per trade
circuit_breaker	Trading halted by circuit breaker	Price movement exceeds 5% in 5 minutes

7. Best Practices for API Usage

7.1. Authentication Best Practices

- Store tokens securely, avoid hardcoding.
- Refresh tokens before expiry.

- Use MFA for sensitive operations.
- Revoke tokens on user logout.

7.2. Efficient Data Retrieval

- Use WebSocket for real-time data to reduce polling.
- Specify minimal fields in requests (e.g., ticker, timeframe).
- Paginate large datasets with limit/offset.
- Cache responses where applicable.

7.3. Handling Rate Limits

- Monitor Retry-After header in 429 responses.
- Implement exponential backoff for retries.
- Distribute requests evenly to avoid bursts.
- Use WebSocket for high-frequency updates.

7.4. Error Handling Strategies

- Parse error messages for actionable details.
- Retry on 429 and 503 errors with backoff.
- Log errors with request_id for debugging.
- Gracefully degrade on API unavailability.

8. Missing Information and Best Practices

8.1. Missing Information

- Specific IBKR API rate limit details for edge cases.
- Exact compliance requirements for audit logs.
- News feed provider specifications.
- Detailed trader personas for API usage patterns.
- Performance benchmarks for API endpoints.

8.2. Best Practices Applied

- **Rate Limits:** Set to 100 requests/minute based on typical trading system standards. **Reason:** Balances performance and fairness.
- **Compliance:** Included immutable audit logs for 7 years. **Reason:** Aligns with SEC/GDPR requirements.
- **News Endpoint:** Implemented as placeholder for Reuters/Bloomberg. **Reason:** Prepares for future integration with reliable sources.
- **Error Handling:** Standardized JSON error format. **Reason:** Improves client-side parsing and debugging.
- **Authentication:** Used OAuth2 with JWT and MFA. **Reason:** Industry-standard for secure API access.

9. Implementation Notes

9.1. Development Environment

- **Framework:** FastAPI for REST and WebSocket APIs.
- **Language:** Python 3.10.
- **Dependencies:** Pydantic for validation, ib_insync for IBKR integration.
- **Tools:** VS Code, Poetry for dependency management.
- **Testing:** Postman, pytest for unit tests.

9.2. Testing the API

- **Unit Tests:** Validate endpoint logic with pytest.
- **Integration Tests:** Test endpoint interactions with IBKR API.
- **Load Tests:** Simulate 100 users with Locust.
- **Security Tests:** OWASP ZAP for vulnerabilities.
- **Mocking:** Use mock IBKR API for offline testing.

9.3. Deployment Considerations

- **Environment:** Lenovo Legion Pro 5, Docker Compose.
- **CI/CD:** GitHub Actions for build/test/deploy.
- **Monitoring:** Prometheus for metrics, Grafana for dashboards.

- **Logging:** Structured JSON logs in DuckDB.
- **Rollback:** Revert to previous Docker image on failure.

10. Glossary

- **ATS:** Algorithmic Trading System.
 - **IBKR:** Interactive Brokers.
 - **OHLCV:** Open, High, Low, Close, Volume.
 - **VW SMA:** Volume-Weighted Simple Moving Average.
 - **MFI:** Money Flow Index.
 - **ATR:** Average True Range.
 - **FastAPI:** Python API framework.
 - **JWT:** JSON Web Token.
-

11. Appendices

11.1. Sample API Requests

Get Market Data:

```
curl -H "Authorization: Bearer <access_token>" \
  "https://ats.local:8000/market-data?ticker=AAPL&timeframe=1m"
```

Place Order:

```
curl -X POST -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" \
  -d
'{"ticker":"AAPL","action":"BUY","quantity":100,"price":150.90,"type":"LIMIT","account_id":"DUK221396"}' \
  "https://ats.local:8000/orders"
```

WebSocket Connection (JavaScript):

```
const ws = new WebSocket('wss://ats.local:8000/ws/market-data', [], {
  headers: { Authorization: 'Bearer <access_token>' }
});
ws.onopen = () => {
  ws.send(JSON.stringify({
    action: 'subscribe',
    tickers: ['AAPL'],
    timeframe: '1m'
  }));
};
ws.onmessage = (event) => console.log(JSON.parse(event.data));
```

11.2. OpenAPI Specification (Excerpt)

openapi: 3.0.3

info:

title: ATS API

version: 1.0.0

paths:

/market-data:

get:

summary: Retrieve real-time market data

parameters:

- name: ticker

in: query

required: true

schema:

type: string

- name: timeframe

in: query

required: true

schema:

```

    type: string
responses:
  '200':
    description: Successful response
    content:
      application/json:
        schema:
          type: object
          properties:
            data:
              type: array
              items:
                type: object
                properties:
                  ticker: { type: string }
                  exchange: { type: string }
                  timestamp: { type: string, format: date-time }
                  open: { type: number }
                  high: { type: number }
                  low: { type: number }
                  close: { type: number }
                  volume: { type: integer }
            metadata:
              type: object
              properties:
                timestamp: { type: string, format: date-time }
                request_id: { type: string }
                total_count: { type: integer }

```

11.3. References

- FastAPI: <https://fastapi.tiangolo.com/>
- IBKR API: <https://interactivebrokers.github.io/tws-api/>
- OAuth2: <https://oauth.net/2/>
- WebSocket: <https://websocket.org/>
- SEC Regulations: <https://www.sec.gov/>